

Meeting summary — DINOv2 fMRI foundation model

2026-06-14

Meeting summary — 2026-06-14

Attendees: Dan, Yoni, Ariel. **Subject:** technical direction for the next iteration of the DINOv2-based fMRI foundation model.

This document records the decisions made during the meeting. The current PDF describing the existing patchify module (PATCHIFY_ARCHITECTURE.pdf, also in the repo) remains the reference for what is already implemented.

1. Pretraining datasets expansion

Motivation

The current pretraining uses **HCP-Young Adult only** (~1 200 subjects, young healthy adults aged 22–35). This is far from the demographic targeted downstream (ADNI, aged 55–90). Recent SOTA work (notably SLIM-Brain, arXiv 2512.21881) shows that **multi-source pretraining with a modest scale (~4 k sessions)** outperforms single-source UK Biobank pretraining at much larger scale.

Decision: which datasets to add

Extend pretraining to include three additional cohorts, biased toward aging populations.

Dataset dimensions

Dataset	Subjects	TR (s)	Timepoints T	Scan length	Access mode	Rationale
HCP-YA (existing)	1 200	0.72	1 200	14.4 min	already on Moriah	Young healthy baseline
HCP-Aging	~700	0.8	~470	6.3 min	NDA controlled (1–3 months)	Aging healthy adults — closes the age gap with ADNI
OASIS-3	~1 098	2.2	~180	6.6 min	registration + proposal (1–7 days)	Aging + AD diagnosed (longitudinal CDR)
AOMIC	~440	2.0	~480	16 min	open access	Scanner / acquisition diversity (Amsterdam)
PIOP1+2	~3 440					Comparable to SLIM-Brain's 4 129 sessions
Total						

Decision: mixing datasets of different T — padding to the longest

All datasets are padded along the temporal axis to match the largest T in the corpus, which is $T_{\max} = 1200$ (HCP-YA). Shorter scans are zero-padded at the end of the sequence. This preserves the longest temporal context available (HCP-YA at full resolution) without truncation.

Concrete padding amounts:

Dataset	Original T	Padded to	Padded frames	Fraction padded
HCP-YA	1 200	1 200	0	0%
HCP-Aging	~470	1 200	~730	61%
OASIS-3	~180	1 200	~1 020	85%
AOMIC PIOP1+2	~480	1 200	~720	60%

The masking strategy (Section 2) will need to respect the padding mask so that padded frames are never used as prediction targets and never contribute to the loss.

Open question 1 — data leakage in SSL pretraining

Whether the foundation model can be pretrained on the **full data** of each dataset (no held-out subjects) without introducing data leakage when those same datasets are then used downstream. The argument in favour: SSL never sees labels, so it cannot memorise the target. The argument against: SSL learns subject-specific anatomical representations, and reusing the same subjects downstream may inflate metrics through subject familiarity rather than genuine generalisation. SOTA practice is inconsistent — Brain-JEPA and SLIM-Brain both use fixed train/val/test splits at the subject level even for SSL pretraining, while BrainLM uses 80% of UK Biobank for SSL pretraining and evaluates on the held-out 20%. **To be decided at next meeting.**

Open question 2 — TR heterogeneity

Different TRs across datasets (0.72 s to 3.0 s) mean that a fixed temporal window of N tokens corresponds to different physical durations (e.g. 20 timepoints at $TR = 0.72$ s is 14.4 s of brain activity, vs. 60 s at $TR = 3.0$ s). The model would learn different temporal semantics per dataset. Three possible mitigations: 1. **Resample all datasets** to a common TR before padding (e.g. resample everything to $TR = 0.72$ s). 2. **Keep native TR** and accept the inconsistency (simplest, what is currently proposed). 3. **Per-dataset temporal kernel** so that each token spans the same physical duration regardless of TR.

To be decided at next meeting.

2. Augmentation strategy refactor

Current setup

DINOv2 standard multi-crop: 2 global crops + 8 local crops, where each crop is a **random spatial sub-volume** of the brain (random resized 3D crop at scales [0.32, 1.0] for global, [0.05, 0.32] for local). Masking is BeiT-style block masking with uniform sampling of mask ratio in [10%, 50%].

Problems with the current setup

1. **Spatial crops are not semantically meaningful for fMRI.** Cropping a sub-volume of a brain is not analogous to cropping a region of a natural image, since the brain is a fixed anatomical structure rather than a scene containing multiple objects.
2. **BeiT block masking is spatially incoherent in our grid.** The flattened spatial ordering of the $N_{\text{spatial}} = 150$ patches does not respect 3D anatomical neighbourhoods, so the “rectangles” sampled by the BeiT generator can contain spatially disconnected patches.

Decision

Replace the multi-crop strategy with a masking-only strategy on the full image.

Crops

Type	Quantity (hyperparameter)	Content	Masking
Global crop	<code>num_global_crops = 1</code>	Full image (no crop)	0% (no masking)
Local crops	<code>num_local_crops = 3</code>	Full image (no crop)	Independent mask per crop

Masking

- **Type:** random per-token masking (MAE-style), replacing the current BeiT block masking. Each token has an independent probability of being masked.
- **Mask ratio per local crop:** sampled from a truncated Gaussian.
 - Mean: 30%
 - Min: 10%
 - Max: 50%
 - Std: to be determined (suggestion: 10%)
- Each of the three local crops samples its own ratio and its own mask independently.

Temporal augmentation

- **None for now.** Each crop uses the full temporal sequence.
- May be reconsidered later (random temporal sub-window, etc.).

Exposed hyperparameters (config)

```
crops:
  num_global_crops: 1
  num_local_crops: 3
  global_mask_ratio: 0.0
  local_mask_ratio_mean: 0.30
  local_mask_ratio_min: 0.10
  local_mask_ratio_max: 0.50
  local_mask_ratio_std: 0.10      # to validate empirically
  masking_type: "random_per_token" # vs "block_beit"
```

3. Fourier features for spatial positional encoding

Motivation

The fMRI spatial resolution is 2 mm per voxel. After patchify, patches are spaced 18 mm apart (9-voxel patch_size). The transformer must distinguish between spatially adjacent patches, but the current spatial positional encoding is a learned lookup table (pos_spatial, shape 150 × 384) that has no built-in geometric structure: nothing in the encoding tells the transformer that patches close in 3D should have similar representations.

This is the same problem solved by Yoni Choukroun et al. (arXiv 2306.15971, MICCAI 2023) in their HRF Transformer, which uses Fourier features (Tancik et al., NeurIPS 2020 — *Fourier features let networks learn high frequency functions in low dimensional domains*) to encode pairwise distances between neurons and vessels for geometric attention modulation.

How Fourier features work (intuition)

Given a low-dimensional input (e.g. a 3D coordinate (x, y, z) or a scalar distance d), Fourier features map it into a higher-dimensional space using cos/sin at multiple frequencies:

$$\gamma(\mathbf{v}) = [\cos(2\pi\mathbf{B}\mathbf{v}), \sin(2\pi\mathbf{B}\mathbf{v})]$$

where B is a matrix of frequencies (typically sampled from a Gaussian). At high frequencies, two nearby positions (e.g. x = 0.500 vs x = 0.522) get mapped to very different cos/sin values, while at low frequencies they remain similar. The result is a structured signature where neighbours stay measurably close at low frequencies but become distinguishable at high frequencies — exactly the property needed for the transformer to tell adjacent patches apart.

Decision

Replace the learned pos_spatial lookup table with a Fourier-feature-based positional encoding of the 3D patch coordinates.

Implementation (Option B in our discussion)

The change is contained entirely within the PositionEmbedding3D module of dinov2/layers/patch_embed_3d_plus_1d.py. Sketch:

```
class FourierFeatures3D(nn.Module):
    def __init__(self, num_freqs=32, sigma=10.0):
        super().__init__()
        B = torch.randn(num_freqs, 3) * sigma
        self.register_buffer('B', B) # or nn.Parameter to make learnable

    def forward(self, positions): # positions: (N, 3)
        proj = 2 * math.pi * positions @ self.B.t()
        return torch.cat([torch.cos(proj), torch.sin(proj)], dim=-1)

class PositionEmbedding3DFourier(nn.Module):
    def __init__(self, gx, gy, gz, embed_dim, num_freqs=32, sigma=10.0):
        super().__init__()
        self.fourier = FourierFeatures3D(num_freqs, sigma)
        coords = make_grid_coords(gx, gy, gz) # (N_spatial, 3)
        self.register_buffer('coords', coords)
        self.proj = nn.Sequential(
            nn.Linear(2 * num_freqs, embed_dim),
            nn.GELU(),
            nn.Linear(embed_dim, embed_dim),
```

)

```
def get_spatial_pos(self):  
    return self.proj(self.fourier(self.coords)).unsqueeze(0)
```

The temporal positional embedding and CLS positional embedding remain unchanged (still small learned tables).

Hyperparameters

Name	Initial value	Description
num_freqs	32	Number of frequencies in the Fourier mapping
sigma	10.0	Scale of the random Gaussian frequency matrix

Alternative considered (Option A, not retained for the first iteration)

A second option discussed was **geometric attention modulation** in the style of the Choukroun et al. paper: keep the existing positional encoding, but additionally multiply the attention matrix by a learned function $f(d_{ij})$ of the pairwise 3D distance, where d_{ij} is processed through Fourier features + MLP. This is a stronger inductive bias but requires modifying the attention block of the ViT itself, which is a much larger change. We will revisit this option if Fourier positional encoding alone proves insufficient.

4. Implementation roadmap

Order	Task	Files affected
1	Replace <code>pos_spatial</code> learned table by Fourier-feature encoding (Section 3)	<code>dinov2/layers/patch_embed_3d_plus_1d.py</code>
2	Refactor augmentation: 1 global + 3 local crops, full image, per-token Gaussian masking (Section 2)	<code>dinov2/data/transforms_3d.py</code> , <code>dinov2/data/masking.py</code> , <code>dinov2/data/collate.py</code> , <code>dinov2/train/train.py</code> , training configs
3	Download HCP-Aging, OASIS-3, AOMIC PIOP1+2 (Section 1) and add to <code>MixedFMRIDataset</code> with subject-level pretrain/eval split	<code>dinov2/data/fmri_data.py</code> , new dataset classes, JSON splits
4	Train V2 foundation model on the multi-source corpus with the new masking strategy	new config <code>fmri_vits_v2.yaml</code>
5	Linear probe on held-out subjects + public benchmarks (ABIDE, ADHD-200, OASIS-3 held-out)	new probe scripts mirroring <code>probe_adni.py</code>

Open questions to confirm at next meeting

- Subject-level split protocol for OASIS-3 (pretrain vs evaluation). JSON manifest format?
- Value of `local_mask_ratio_std` (suggested 0.10).
- Should the Fourier frequency matrix `B` be **fixed** (Gaussian random, registered as buffer) or **learnable** (`nn.Parameter`)?
- Should `num_global_crops` and `num_local_crops` be varied as part of an ablation, or fixed at 1 and 3?

Reference document

The current patchify module (`PatchEmbed3DPlus1D`) is documented in `PATCHIFY_ARCHITECTURE.pdf` (this repository). The Fourier features will replace the spatial part of `PositionEmbedding3D` in that same module; the rest of the patchify architecture (`Conv3Plus1d`, `_ResBlock3Plus1d`, hierarchical encoder) remains unchanged.

Cited literature

- Choukroun, Y., Golgher, L., Blinder, P., Wolf, L. *Reconstructing the Hemodynamic Response Function via a Bimodal Transformer*. arXiv 2306.15971, MICCAI 2023.
- Tancik, M. et al. *Fourier features let networks learn high frequency functions in low dimensional domains*. NeurIPS 2020.
- SLIM-Brain (anonymous preprint). arXiv 2512.21881, December 2025.
- Brain-JEPA. Dong & Li et al., NeurIPS 2024 Spotlight. arXiv 2409.19407.
- MAE block masking strategy reference, used by BrainLM (ICLR 2024). bioRxiv 2023.09.12.557460.